

# Gas Optimisation Report

## Project 3 — DAO Governance Contract

---

### Section 1 — Optimisations Applied

The following five optimisations were applied across DAOGovernance.sol and Treasury.sol. Each replaces a more expensive pattern with a cheaper equivalent. The table below lists all five for completeness. Section 2 presents the full before/after gas measurement for the primary optimisation.

| # | Optimisation Applied                    | What Replaced                                         | File              |
|---|-----------------------------------------|-------------------------------------------------------|-------------------|
| 1 | Description emitted as event log        | string description stored in struct (SSTORE)          | DAOGovernance.sol |
| 2 | Custom errors (error X())               | require() with string messages                        | DAOGovernance.sol |
| 3 | calldata for data + description params  | memory keyword — causes heap copy                     | DAOGovernance.sol |
| 4 | Storage vars cached to memory in vote() | Direct p.startTime, p.endTime etc. reads (SLOAD each) | DAOGovernance.sol |
| 5 | Modifier extracted to internal function | Inline modifier body duplicated per call site         | Treasury.sol      |

### Section 2 — Primary Optimisation: Description as Event Log

---

#### 2.1 The Problem

In the unoptimised contract, the Proposal struct included a **string description** field stored directly in contract storage. Writing a string to storage triggers the **SSTORE** opcode. The EVM charges **20,000 gas** for writing to a new storage slot. A 500-character description costs approximately 40,000 gas. This cost is paid by every user who creates a proposal — making it the single most expensive avoidable operation in the contract.

#### 2.2 Before — Unoptimised Code

The description field was stored inside the Proposal struct and written to contract storage on every createProposal() call:

```
// BEFORE: description stored in struct — SSTORE on every proposal
struct Proposal {
```

```

uint256 id;
uint256 snapshotId;
uint256 startTime;
uint256 endTime;
uint256 yesVotes;
uint256 noVotes;
bool executed;
bool canceled;
address creator;
address target;
bytes data;
string description; // <-- stored on-chain, costs ~20,000 gas SSTORE
}

function createProposal(
    address target,
    bytes memory data,          // memory - heap copy
    uint256 votingDelay,
    uint256 votingPeriod,
    string memory description // memory - heap copy
) external returns (uint256) {
    // description written to storage here - costs ~20,000 gas
}

```

## 2.3 After — Optimised Code

The description field was removed from the struct entirely. Instead it is emitted in the **ProposalCreated** event. The function parameters also changed from **memory** to **calldata** to avoid copying bytes and string onto the heap:

```

// AFTER: description NOT in struct - emitted as event log only
struct Proposal {
    uint256 id;
    uint256 snapshotId;
    uint256 startTime;
    uint256 endTime;
    uint256 yesVotes;
    uint256 noVotes;
    address creator; // packed with bools in same slot
    bool executed;
    bool canceled;
    address target;
    bytes data;
    // description field removed entirely
}

function createProposal(
    address target,

```

```

    bytes calldata data,          // calldata — no heap copy
    uint256 votingDelay,
    uint256 votingPeriod,
    string calldata description // calldata — no heap copy
) external returns (uint256) {
    // description emitted as event — costs ~375 gas
    emit ProposalCreated(proposalCount, msg.sender, target,
                        startTime, endTime, description);
}

```

## 2.4 Why This Works

Event logs are stored in transaction receipts — not in contract storage. The EVM charges approximately **375 gas** to emit a 32-byte event topic, compared to **20,000 gas** for writing a new storage slot with SSTORE. The description is still permanently accessible on-chain via event logs — it is simply not queryable by the contract itself at runtime. Since no contract function ever needs to read the description back from storage, this is a lossless optimisation.

This also aligns with the IPFS pattern mandated in the project rubric: the frontend uploads the full proposal text to IPFS, receives a 46-character CID (content identifier), and passes that CID as the description argument. The on-chain CID is the proof of authenticity — anyone can fetch the content from IPFS and verify it matches the hash.

## 2.5 Gas Cost Comparison

| Storage Method            | Gas Cost    | Accessible How?      | Contract Readable? |
|---------------------------|-------------|----------------------|--------------------|
| string in struct (SSTORE) | ~20,000 gas | Contract storage     | Yes                |
| Event log — optimised     | ~375 gas    | Transaction receipts | No (not needed)    |
| IPFS CID in event log     | ~512 gas    | IPFS + receipts      | No (not needed)    |

## Section 3 — Before / After Gas Numbers

Both reports generated using: **forge test --gas-report** on an Anvil local node. 27/27 tests passed in both versions. The highlighted row shows the primary optimisation target.

### 3.1 Comparison — DAOGovernance Function Calls

| Function         | Before — Avg Gas (Unoptimised) | After — Avg Gas (Optimised) | Gas Saved | Change |
|------------------|--------------------------------|-----------------------------|-----------|--------|
| createProposal() | 307,620                        | 287,944                     | 19,676    | -6.4%  |
| vote()           | 73,865                         | 75,908                      | -2,043    | +2.8%  |

|                       |           |           |          |               |
|-----------------------|-----------|-----------|----------|---------------|
| executeProposal()     | 72,439    | 74,496    | -2,057   | +2.8%         |
| cancelProposal()      | 28,112    | 28,364    | -252     | ~0%           |
| Deployment (one-time) | 1,487,998 | 2,424,303 | -936,305 | <b>+62.9%</b> |

**Key result:** createProposal() saves **19,676 gas per call** — a 6.4% reduction. This directly corresponds to removing the SSTORE for the description field (~20,000 gas). The deployment cost increased by 936,305 gas because custom errors, struct packing, and calldata handling add bytecode. This is a one-time cost paid by the deployer. The optimised contract breaks even after approximately **48 proposals** and delivers permanent savings to all users thereafter.

## Section 4 — Full Gas Reports (forge test --gas-report output)

### 4.1 Before — Unoptimised

| Function         | Min     | Avg     | Median  | Max     | # Calls |
|------------------|---------|---------|---------|---------|---------|
| cancelProposal   | 24,020  | 28,112  | 30,132  | 30,132  | 5       |
| createProposal   | 273,927 | 307,620 | 308,127 | 330,528 | 23      |
| executeProposal  | 28,700  | 72,439  | 77,493  | 108,866 | 11      |
| hasVoted         | 2,639   | 2,639   | 2,639   | 2,639   | 2       |
| proposalCount    | 2,586   | 2,586   | 2,586   | 2,586   | 3       |
| proposals        | 27,044  | 27,044  | 27,044  | 27,044  | 5       |
| quorumPercentage | 2,432   | 2,432   | 2,432   | 2,432   | 1       |
| updateQuorum     | 23,769  | 30,843  | 26,916  | 45,772  | 4       |
| vote             | 26,221  | 73,865  | 72,327  | 89,427  | 30      |

### 4.2 After — Optimised

| Function        | Min     | Avg     | Median  | Max     | # Calls |
|-----------------|---------|---------|---------|---------|---------|
| cancelProposal  | 23,987  | 28,364  | 30,574  | 30,574  | 5       |
| createProposal  | 254,250 | 287,944 | 288,450 | 310,862 | 23      |
| executeProposal | 28,981  | 74,496  | 81,108  | 111,658 | 11      |
| hasVoted        | 3,159   | 3,159   | 3,159   | 3,159   | 2       |
| proposalCount   | 2,491   | 2,491   | 2,491   | 2,491   | 3       |

|                  |        |        |        |        |    |
|------------------|--------|--------|--------|--------|----|
| proposals        | 26,366 | 26,366 | 26,366 | 26,366 | 5  |
| quorumPercentage | 2,536  | 2,536  | 2,536  | 2,536  | 1  |
| updateQuorum     | 24,179 | 34,640 | 27,466 | 59,451 | 4  |
| vote             | 30,660 | 75,908 | 74,284 | 91,384 | 30 |